Concurrency-related Flaky Test Detection in Android Apps*

Zhen Dong
National University of Singapore
zhen.dong@comp.nus.edu.sg

Abhishek Tiwari
National University of Singapore
dcsabhi@nus.edu.sg

Xiao Liang Yu
National University of Singapore
xiaoly@comp.nus.edu.sg

Abhik Roychoudhury
National University of Singapore
abhik@comp.nus.edu.sg

arXiv:2005.10762v3 [cs.SE] 21 May 2021

ABSTRACT

Validation of Android apps via testing is difficult owing to the presence of flaky tests. Due to non-deterministic execution environments, a sequence of events (a test) may lead to success or failure in unpredictable ways. In this work, we present an approach and tool FlakeScanner for detecting flaky tests through systematic exploration of event orders. Our key observation is that for a test in a mobile app, there is a testing framework thread which creates the test events, a main User-Interface (UI) thread processing these events, and there may be several other background threads running asynchronously. For any event *e* whose execution involves potential non-determinism, we localize the earliest (latest) event after (before) which *e* must happen. We then efficiently explore the schedules between the upper/lower bound events while grouping events within a single statement, to find whether the test outcome is flaky. We also create a suite of subject programs called FlakyAppRepo to study flaky tests in Android apps. Our experiments on subject-suite FlakyAppRepo demonstrate the efficacy of our flaky test detection. Our work is complementary to existing flaky test detection tools like Deflaker which check only failing tests. FlakeScanner can detect flaky tests among passing tests, as shown by our approach and experiments.

1 INTRODUCTION

GUI testing is notoriously difficult. A GUI test is brittle, a little change to underlying code could break the test. Even worse is a GUI test that passes sometimes and fails sometimes, without any change in the code, tests, or environment. These tests are called *flaky tests*. Flaky tests occur often due to non-deterministic execution environments. Take an Android app, for example, that needs to connect to the internet. Based on the network stability, connection may take less or more time. If this kind of uncertainty is not properly dealt with, a test may pass when a connection is fast and fail for a slow connection, manifesting the flaky behavior.

Flaky tests frustrate developers and significantly hinder the testing automation. Developers often run tests to verify that their latest changes to a code repository did not break any previously working functionality, i.e., *regression testing*. Ideally, a test failure would be due to the code changes and developers can focus on debugging these changes. Unfortunately, some test failures are not due to the code change but due to flakiness. Identifying whether a failure is due to code changes or flaky tests may require tremendous efforts and slow down software development. Google statistics [2] show that in practice 84% of transitions from pass to fail involve a flaky test in regression testing, which causes significant drag on software engineers. Facebook recently established the detection of flaky tests as a top research problem for software testing [4].

Today, the technology to detect flaky tests is not mature. Developers face a considerable amount of failures due to flaky tests in regression testing [12]. They struggle to distinguish them from the failures that are due to a recently introduced regression. A typical way to detect flaky tests is to rerun failing tests repeatedly (called *RERUN*). Specifically, developers rerun each failing test multiple times after witnessing the failure. If some rerun passes, the test is marked flaky, otherwise it is marked as unknown. Despite being simple and often used in practice, RERUN is unreliable. Flaky tests are nondeterministic by definition, so there is no guarantee that the outcome of a flaky test will change within multiple reruns. Besides, RERUN is extremely costly: A GUI test typically executes much slower and may take minutes for a single run.

DeFlaker [5] detects flaky tests with differential coverage analysis. A test is marked as flaky if the test fails and at the same time it did not cover any of latest code changes during regression testing. Unfortunately, DeFlaker only works on failing tests and cannot examine whether tests that passed are flaky. Flaky tests that pass in the current regression testing cannot be detected; thus, they remain in the test suite.

In this work, we propose a proactive approach to expose flaky tests of Android apps. Our approach takes a passing test as input and executes it in different possible execution environments. If there is an environment in which the test fails, the test is deemed flaky. With this approach, flaky tests can be exposed and quarantined from a test suite before regression testing occurs. Thus, failures due to test flakiness can be avoided in regression testing. This is with the goal of reducing developer frustrations when they "discover" that a test that they have been using all along is flaky! Instead, we check whether a test is flaky *before* introducing it into the test-suite.

Observation. Android adopts the single GUI thread model in which events are processed by a UI thread sequentially. To avoid blocking the UI thread for responsiveness, a long-running task, such as accessing the internet, is offloaded to an async thread. Once the task is completed, the async thread updates the result by submitting an event (called an *async event*) to the UI thread, which results in event racing. Under this model, event execution order varies from run to run due to non-deterministic execution environments. An app might behave normally for most orders but act in an unexpected way for certain orders. This eventually leads to a phenomenon where a test passes sometimes and fails at other times.

*The extended version is available at https://github.com/FlakyTest/FlakyTests and will appear on ESCE/FSE2021.

Insight. A flaky test can be exposed by exploring the space of event execution orders that may result from different execution environments. Instead of exhaustively running a test in all possible execution environments, we examine a test by exploring the space of feasible event execution orders to check whether there exists an event execution order in which the test fails. A test is deemed flaky if a failure is detected during exploration. Our insight is that Android apps often assign environment-related tasks such as accessing background service or the internet to async threads. App behaviors in different execution environments can be explored by scheduling events.

Challenges. Computing the space of possible event execution orders is difficult. As explained above, event execution orders are determined when async threads submit the async events at runtime. A test run may involve many async threads which are launched from different layers including Android framework, third-party libraries, and the app under test itself. This poses a challenge to track them. Moreover, computing the possible event execution orders involves resolving event dependencies due to thread synchronization. A GUI test often runs in a separated thread maintained by a testing framework such as Espresso [1]. Thus, a thorough analysis of the interleavings between the app, the Android framework, and the testing framework is required. Existing techniques for Android apps focus mainly on finding specific concurrency bugs, e.g., CAFA [17] is limited to finding race errors due to use-after-free violations. Additionally, they only support app analysis and provide no support for testing framework.

Solution. We propose a system-level dynamic analysis to resolve thread synchronization dependencies. We run apps in the debug mode such that threads in the whole Android runtime can be monitored and controlled. Thread synchronization dependencies can be resolved by manipulating threads, e.g., suspending a thread to observe others. Besides, a GUI test consists of multiple statements with various GUI operations such as a button click. These operations create multiple events to complete their execution, which leads to a huge space of possible event execution orders. Enumerating all of them is costly, considering a GUI test may run slowly and take several minutes. To address this, for each statement, such as a button click, we group the events it generates. Finally, we schedule the async events between these groups.

Experiments. We evaluate our approach on FlakyAppRepo which contains 28 widely-used Android apps including Firefox app from Mozilla. We detected 19 out of 24 previously known GUI related flaky tests. We analyzed their root causes and categorized them into three categories. Additionally, we discovered 245 flaky tests that were previously unknown.

Contributions. Our contributions can be summarized as follows.

- We propose a proactive approach to expose flaky tests in a test suite. A flaky test can be exposed and quarantined before regression testing so that failures due to flaky tests can be avoided. It is the first approach that automatically detect flaky GUI tests for Android apps.
- We develop a technique FlakeScanner which can control threads launched from different layers including apps, Android framework and testing framework, and perform a system level dynamic analysis to precisely resolve dependencies between events.
- We collect a subject-suite that contains 28 widely-used apps with GUI tests that are from developers, called FlakyAppRepo. To facilite future research on flaky tests, we make our tool FlakeScanner and subject-suite FlakyAppRepo publicly available at: *https://github.com/FlakyTest/FlakyTests*

2 ANDROID CONCURRENCY AND TESTING

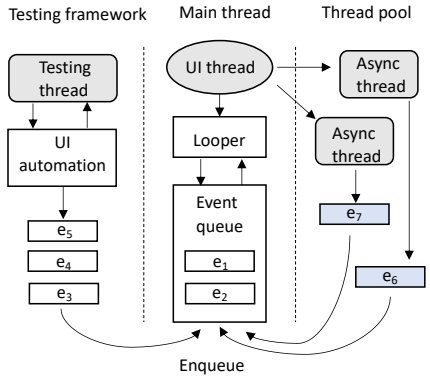


Figure 1: Android concurrency model & testing framework.

Figure 1 depicts the Android hybrid event-driven concurrency model as well as the Android GUI testing framework. Every app has a main thread (also called *UI thread*), which maintains an event queue and a looper associated with the event queue. UI or system events that are generated by users or the system are added into the event queue. The looper dequeues events in a sequential order and dispatches them to corresponding handlers for processing. Android adopts the single-UI-thread model where only the main thread can access the GUI objects. To prevent non-responsive threads from blocking the GUI, long-running tasks such as network access are offloaded to async threads. Once these tasks are finished, async threads post an event marked in blue in Figure 1 to the main thread, which updates results to GUI objects. However, this concurrency model can lead to event racing. As shown in Figure 1, the main thread and async threads run concurrently. When an async thread finishes a task and posts an async event is non-deterministic, depending on the current execution environment. Consequently, the order of events processed by the main thread is non-deterministic as well. In the example shown in Figure 1, there are multiple possible orders which might occur in the execution such as $<e_1, e_2, e_6, e_3, e_4, e_5, e_7>$ and $<e_1, e_2, e_3, e_4, e_5, e_6, e_7>$.

Android provides testing frameworks for developers to write GUI tests, which simulate user interactions to exercise app functionalities. GUI tests run on physical devices or emulators and interact with UI interface to generate events. To achieve more reliable tests, testing frameworks provide a set of mechanisms to synchronize test automation interactions with the user interface. For instance, when method `onView()` is invoked in a test, Espresso waits to perform the corresponding UI action or assertion until the event queue is empty and some async threads (e.g., AsyncTask instance) are terminated and user-defined resources are idling.

3 A MOTIVATING EXAMPLE

In this section, we use a simple example to explain how a flaky test occurs in Android apps and the challenges in detecting such flaky tests. The example comes from the *RapidPro Surveyor* app for Android and related code snippets are shown in Listings 1- 3. As we see, the test (Listing 1) first launches an activity (Listing 2) that is used to capture location data of the Android device. When created, the activity connects to the Google API client (line 4, Listing 2) and requests location data of the device. During the connection process, Google API client creates a few worker threads (refer Listing 3) to complete the connection process. Finally, the Google API client accesses the location data and sends it to the activity. Then the test clicks a button on the activity to obtain the location data from the activity. In the end, the test checks the obtained data is not `NULL`.

Despite being simple, the test is a flaky test. As mentioned before, the test is executed in a testing thread, and the activity runs on the UIThread of the app, and the operation that the Google API client obtains location data is executed on an async thread. Although the test uses `onView()` to synchronize GUI operations, the testing thread cannot synchronize with the async thread that fetches the location data. Thus, the async thread might update the location data to the activity before or after the testing thread checks the location data. If the checking occurs before the activity receives the location data, the test fails. Otherwise, the test passes. This leads to a phenomenon that the test passes for some times and fails for other times.

Detecting such flaky tests is difficult. First of all, a flaky test is "hiding" in the test suite and may pass in most execution environments. There is nothing different from other passing tests unless the test fails in the execution. The existing techniques of detecting flaky tests such as DeFlaker [5] apply on failing tests and cannot examine whether a passing test is flaky. Although many existing techniques can detect concurrency bugs, they face challenges to detect a flaky test. As shown in the example, a test in Android apps is often run by a testing framework and many threads might come from the Android system. Detecting flaky tests requires to analyze not only app under test but also the testing frameworks. However, existing techniques typically focus on app analysis. For instance, DroidRacer [25] records execution traces and detects data race in apps by offline analyzing collected execution traces. ERVA [18] takes a data race report which are generated by other tools like DroidRacer to verify whether the reported data race is true positive. AsyncDroid [21] detects bugs in an app by exploring alternative execution orders of event handlers that are created by the app. None of them deals with analysis of testing frameworks and applies to flaky tests detection. This urgent need motivates us to develop a technique that can detect flaky tests for Android apps.

Listing 1: CaptureLocationActivityTest (Espresso Thread)

```
1  @Rule
2  public ActivityTestRule<CaptureLocationActivity> rule = new
       ActivityTestRule<>(CaptureLocationActivity.class);
3  @Test
4  public void capture() {
5    onView(withId(R.id.button_capture))
                          .check(matches(isDisplayed()))
                          .perform(click());
6    Instrumentation.ActivityResult result =
       rule.getActivityResult();
7    assertThat(result.getResultData(), is(not(nullValue())));
8    // ...
9   }
10 }
```

Listing 2: CaptureLocationActivity (Main Thread)

```
1  @Override
2  protected void onCreate(Bundle bundle) {
3    // ...
4    connectGoogleApi();
5    // ..
6  }
7  protected void connectGoogleApi() {
8    googleApiClient = new GoogleApiClient.Builder(this)
9                          .addConnectionCallbacks(this)
10                         .addOnConnectionFailedListener(this)
11                         .addApi(LocationServices.API)
12                         .build();
13
14   googleApiClient.connect();
15 }
```

Listing 3: Zaau (Worker Thread)

```
1  //Worker thread asynchronous to the main thread
2  @WorkerThread
3  public void run() {
4    zaak.zac(this.zagj).lock();
5    try {
6      if (!Thread.interrupted()) {
7      this.zaan();
8      return;
9      }
10   // ...
11 }
```

4 OVERVIEW

Consider a GUI test T consisting of a sequence of program statements $<s_1, s_2, s_3, s_4, s_5, ...s_m>$ in Figure 2 (a), its one possible execution trace is E that consists of a sequence of executed events $<e_1, e_2, ...e_n>$, and e_3 is an async event (i.e., e_3 is generated by an async thread). For simplicity, we assume only one async event is generated for this test (in general, many async events are generated in a single test execution, which is considered in our approach). As previously stated, event execution order of the test might change for a different run due to non-deterministic execution environments. For instance, event e_3 might be executed before e_2 or after e_7, depending on how much time is needed for its corresponding thread to complete the task and post the event. Our goal is to compute how many possible event execution orders there are for this test and check whether there exists one order in which the test fails. If a test failure is detected, the test is deemed flaky.

Apparently, the position of e_3 in the sequence of events is uncertain for a different test run. However, the space of possible positions of e_3 should be constrained between certain two events due to dependencies resulted from thread synchronization. We assume the

two events are e_u and e_v. So e_3 cannot be executed earlier than e_u or later than e_v, no matter how the execution environment changes (e.g., network connection becomes slow). If e_u and e_v are localized, computing possible event execution orders for the test can be achieved.

Computing space of event execution orders. In event-driven programming, an event is designed for communication among multiple components and well-encapsulated. Event dependencies are typically handled over to other components like event handlers. Thus, it is challenging to identify such dependencies by capturing and analyzing events themselves. Our idea is to link events to statements in a test since all events are triggered by a test. We execute a test statement by statement and record all events triggered by each statement and build a map between them as shown in Figure 2 (b). As we see, e_3 is triggered by statement s_2. Consequently, e_3 cannot be executed earlier than the first event that is triggered by s_2. Thus, lower bound e_u of the async event e_3 can be identified, i.e., e_2. Localizing upper bound e_v of event e_3 involves identifying which events depend on e_3, i.e., events that occur only after e_3 is processed. Testing frameworks typically use thread synchronization to guarantee an event occurs before another. For instance in an Espresso test, a statement that invokes `onView()` method waits until specified threads or resources are idling; otherwise it refuses to be executed. Therefore, event dependency analysis requires to resolve thread synchronization dependencies. As mentioned earlier, traditional program analysis faces challenges to perform such analysis because Android testing often involves many threads from third-party libraries, Android framework, and testing framework. Existing analysis techniques hardly overcome those obstacles, as they are often restricted to analyzing the app code.

Addressing this challenge, we propose a *what-if* analysis. Specifically, after the test is launched, we hook the async thread that posts e_3 at runtime, and suspend the async thread and let other threads free to go. At the same time, we monitor the testing thread and check at which statement of the test it stops and waits for the hooked thread to be completed. Suppose that the testing thread stops at statement s_5, we consider operations in s_5 depends on e_3 and these operations will not be executed until e_3 is processed. Thus, the first event e_9 triggered by s_5 is upper bound of e_3, that is, e_3 has to be executed before e_9. The idea behind our approach is what if it takes forever to compute e_3, operations in a test that depend on e_3 will not be triggered due to thread synchronization and those that do not depend on e_3 will be executed. So, events that depend on e_3 are identified dynamically.

Reducing space of event execution orders. Since the position of e_3 is between e_2 and e_9, possible event execution orders for the test can be calculated by moving e_3 one position at a time, until reaching e_9, for instance, $<e_1, e_2, e_4, e_3, e_5, e_6, ...e_n>$. In practice, the space of event execution orders can be huge because one GUI operation often triggers multiple events at runtime, *e.g.*, one click would generate "click down" and "click up" events. One test may trigger hundreds of events, which leads to a huge space of possible event execution orders. Exploring all of them is costly considering a GUI test runs slowly. Thus, we only consider event execution orders in which e_3 is located before the first event of a statement, i.e., between e_5 and e_6, and between e_8 and e_9 shown in Figure 2(b). This is reasonable because app behavior is more likely to be influenced when an async event is executed after the execution of a statement is completed.

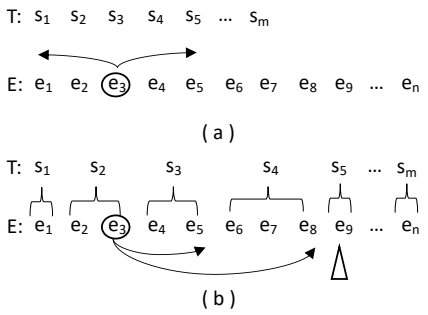


Figure 2: Computing event execution orders of a test.

Scheduling events. Suppose, we are exploring an event execution order which in e_3 is executed prior to e_9 (see Figure 2 (b)). We first query the map between events and statement which is previously generated and identify which statement triggers e_9 (in this case it is statement s_5). Once the test run is launched, we hook the async thread that posts e_3 and suspend the thread such that the event e_3 cannot be posted. At the same time, we monitor the testing thread and check which statement is being executed by querying the program counter in the Android runtime. When the program counter reaches s_5, we suspend the testing thread and free the async thread that we suspended earlier. After the async thread finishes the task, we free the testing thread to run. In this way, e_3 can be executed immediately before the first event e_9 of statement s_5.

5 METHODOLOGY

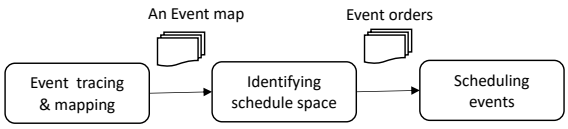


Figure 3: workflow of exposing flaky tests through systematic schedule exploration.

Figure 3 shows the workflow of our approach. Given a test and app under test, it performs a concrete execution to trace events that the test generates and builds a map between statements in the test and events that are triggered by these statements. Then the approach executes the test multiple times to compute possible schedules for async events and generates a set of event orders that might occur in execution environments. Finally, we explore these possible event execution orders for the test. During the exploration, if a test failure is detected, the test is identified as a flaky test.

5.1 Event tracing and mapping

Event tracing is often used in dynamic analysis of Android apps. It can be achieved by simply logging events that are generated at

Algorithm 1: Event tracing and mapping.

```
1  Procedure runTest(App A, Test T, Android ART)
2      launchApp (A, ART);
3      ARTHandler ← attachHandler (A, ART);
4      List ← ∅ ; // storing pairs of a statement and events
5      launchTest (A, T, ART);
       // UI thread's event queue
6      eventQ ← getEventQ(ARTHandler);
7      for s in T do
8          runStatement(ARTHandler, s);
9          n ← getLineNum(ARTHandler, s);
10         while True do
11             E⟨isAsync, sq_m⟩ ← getEvent(ARTHandler);
12             if E != Null then
13                 List ← List ∪ {⟨isAsync, sq_m, n⟩}
14             else
15                 if isEmpty(eventQ) then
16                     break ;
17                 end
18             end
19         end
20     end
21     return List
```

Algorithm 2: Event scheduling exploration.

```
1  Procedure explore(App A, Test T, Android ART, EventMap List)
2      for e in List do
3          if isAsync (e) then
4              launchApp (A, ART);
5              ARTHandler ← attachHandler (A, ART);
6              launchTest (A, T, ARTHandler);
7              TH_test ← getTestingThread (ARTHandler);
8              TH_async ← hookAsyncThread (ARTHandler);
9              suspend (ARTHandler, TH_async) ;
10             while True do
11                 if isWaiting (TH_test) then
12                     break ;
13                 end
14             end
15             m ← getLineNum (ARTHandler, Th_test);
16         end
           // m is set as upper bound along with e
17         setUpperBound (e, m) ;
           // update event e (with upper bound m) in List
18         updateEvent (List, e) ;
19         break ;
20     end
21     return List
```

runtime. However, such techniques cannot fulfill our task. Event information (*e.g.*, event id) produced in logs is dynamically generated at Android runtime and changes in a different run. Our approach requires an event identifier which can be used to identify an event across different test runs. Async events that are identified during event tracing need to be hooked and scheduled in runs which are performed for event order exploration. This poses a challenge for existing techniques.

Event identification. We identify an event based on interactions between the event and app under test at runtime. Two events that are triggered in different test runs are considered as an identical program behavior if: (1) they are triggered by the same test statement; (2) they are processed by a same sequence of methods at runtime. For instance, a `pressDown` event is associated with an identifier which is constructed with line number of the statement that triggers the event and signatures of a sequence of methods that process the event. This practice of event identification comes from our investigation of the Android framework. Events are widely used for thread communication and managed by `Handlers` associated with threads. Events are dealt via different `Handlers` according to where events come from.

Tracing and mapping. Algorithm 1 outlines the procedure of event tracing and mapping. It first launches app under test and takes control of Android runtime in which the app runs with a module called *ARTHandler*. ARTHandler runs the input test T in the testing thread and executes statements one by one. When one statement is executed, ARTHandlers monitors the event queue of the UI thread and hooks injected events. For each event, ARTHandler records the tuple $\langle isAsync, sq_m \rangle$ where *isAsync* denotes whether it is an async event and sq_m denotes the signatures of a sequence of methods that have processed the event. This tuple, along with the line number of the statement that is being executed forms the identifier of the event, which is stored in a list. As stated before, a statement in the test might launch long-running tasks which are executed in async threads. Async events might take long time to be posted. To not miss async events that are triggered by a single statement, we keep hooking events until two criteria are satisfied: (a) there are no new events and (b) the event queue of the UIThread is empty, which often indicates the system is not running tasks. This practice is also used in the Espresso testing framework. A map between statements and events is stored in *List* and returned.

5.2 Identifying event schedule space

To compute possible event execution orders, we perform a what-if dynamic analysis to resolve event dependencies that are caused by thread synchronization in apps and testing frameworks. Algorithm 2 shows the procedure of resolving event dependencies. It takes the event trace *List* generated in the previous step as input. For each async event e_i in *List*, the algorithm launches the test and starts to hook event e_i. Once hooked, the algorithm suspends thread that posts e_i such that e_i can be posted. Meanwhile, it keeps checking status of the testing thread. If the status of the testing thread is `WAITING`, it considers the testing thread is performing thread synchronization with threads in the app and waiting for e_i to be executed. Thus, we consider the statement s that is being executed in the testing thread attempts to trigger an event (saying e_j) which depends on e_i. Therefore, the schedule space of e_i is bounded by e_j, i.e., the first event that is triggered by s. So statement s is identified as the upper bound of schedule space of async event e_i. Statement s is recorded and set as the upper bound of event e_i. When the upper

bound is set, e_i is restored to $List$. In the end, schedule spaces for all async events in $List$ are identified and recorded.

5.3 Scheduling events

Schedule space of each async event in the event trace $List$ is identified in previous steps. Now we explore event orders during test execution. An async event e_i can be simply represented by a triple $<id, n, m>$ where n and m are bounds of the schedule space of e_i. Specifically, n is the index of the statement in the test that triggers e_i, and m is the index of the statement that triggers the upper bound event of e_i .

Similar to schedule space identification, we can schedule e_i by operating threads. We first hook event e_i after the test is launched and suspend the thread that posts e_i. Then, we free the testing thread and monitor whether the statement that is being executed is statement m. Once statement m is reached, we suspend the testing thread and free the suspended thread to post e_i. After the async thread is terminated or idling, i.e., event e_i has been posted, we free the testing thread. In such a way, event e_i can be executed prior to statement m. In next test run, we schedule e_i to be executed prior to statement $m-1$, until all statements between n and m are explored. This procedure is repeated for each async event in $List$ so that the space of possible event execution orders can be systematically explored for the test.

5.4 Optimization

In Algorithm 2, each async event requires one test run for schedule space identification. If n async events are generated for a test, we need to run the test n times to identify the space of event execution orders, which is costly. To address this issue, we perform an optimization on schedule space identification. When the schedule space of an async event is identified, instead of terminating the execution, we continue executing the test to identify schedule spaces for subsequent async events so as to reduce the number of test runs. In this case, we may suspend multiple async threads at the same time and release more than one async threads to resolve a thread synchronization dependency, which results in a group of async events having the same upper bound (latest time). We only rerun the test for this case to identify schedule space of each of them. Thus, the total number of test runs can be significantly reduced.

6 IMPLEMENTATION

Our system is implemented in Scala and runs on a computer that connects a physical Android device or an emulator. Unlike existing techniques, it requires no instrumentation on apps or the Android framework, nor any modification to the Android framework, and can be easily adapted to different versions of Android.

Taking control of Android Runtime. We leverage the Android debug mode to control the Android runtime. The Android framework allows to run an app in the debug mode. In this mode, we interact with the Android runtime using ADB to remotely monitor the app state and manipulate the thread executions, e.g., performing the execution step by step.

Hooking events. Android adopts the event-driven model, in which each app has an event queue for storing events that occurred and processes them one by one. In the debug mode, we are allowed to set a breakpoint at method `enqueueMessage()` which is in charge of enqueuing events. Whenever the method is invoked, our system is informed and performs predefined operations such as suspending the event-posting thread. In such a way, our system can hook any event that occurs in the Android runtime before the event is posted into the event queue.

Operating threads. In the debug mode, we are allowed to inspect threads that are running in the Android runtime and check their statues and operate them by sending commands, e.g., sending a command to release a suspended thread. So the UIthread and testing thread can be identified during a test execution. A breakpoint is inserted at each test statement such that we can fully monitor and control the testing thread including querying the index of statement that is being executed and executing step by step. We also can examine the stack frames of a thread to check executed methods in the thread. Such data is used to identify an event.

7 EVALUATION

We perform evaluation on the effectiveness of FlakeScanner in detecting flaky tests that reside in test suites of real world Android apps. Our evaluation aims to address the following research questions:

- Can FlakeScanner examine and detect known *flaky tests*?
- How does FlakeScanner compare with existing techniques in terms of number of detected flaky tests?
- Can FlakeScanner be used to discover new flaky tests in apps?

7.1 Subject apps

Android app testing has been heavily explored. There are various benchmarks used in evaluating the effectiveness of automated testing of Android apps such as AndroTest [10] which contains 68 open source Android projects and the benchmark [30] from Wang et al. which contains 68 industrial Android apps. However, few apps from those benchmarks come with a test suite from developers, let alone GUI tests. On the other hand, test flakiness is an urgent challenge, especially for GUI tests. Unfortunately, there are no Android app benchmarks to support test flakiness research.

Given the pressing need, we developed the first subject-suite FlakyAppRepo which is used to study GUI test flakiness. It contains 28 widely-used Android apps including Mozilla Firefox Lite and WordPress as shown in Table 1. There are more than 5000 Android instrumentation tests from developers that run on physical devices and emulators.

The challenge we face in building this data set is that publicly available Android projects rarely have tests from developers. To overcome this challenge, we collect Android projects with the following strategies. First, we search well-known Android projects like Firefox in Github and select any project in which there exist tests under folder "../src/androidTest". Second, we search the label "@FlakyTest" in Github and on the Google website, and select any Android project in which at least one of instrumented tests is labelled "@FlakyTest". Tests labelled "@FlakyTest" in a test suite are flaky tests reported by developers.

Table 1: Subject Apps

App Name	Version	#LOC	#Stars	Category
Amaze File Manager	3.4.3	92.2k	2.8k	Tools
Youtube Extractor	2.0.0	2.7k	519	Video Players
AntennaPod	1.8.1	102.6k	2.7k	Music & Audio
CameraView	2.6.1	40.5k	2.9k	Photography
Catroid	0.9.69	457.5k	0	Education
City Localizer	1.1	4k	0	Travel & Local
ConnectBot	1.9.6	119.7k	1.4k	Communication
DuckDuckGo	5.43.0	211.3k	1.2k	Tools
Espresso	1.0	17.3k	1.1k	Maps & Navigation
Firefox Lite	2.18	1598.4k	212	Communication
GnuCash	2.4.0	90.1k	1k	GnuCash
IBS FoodAnalyzer	1.2	26.1k	0	Health & Fitness
Google I/O	7.0.14	73.5k	19.6k	Books & Reference
Just Weather	1.0.0	5.9k	65	Weather
KeePassDroid	2.5.3	159.7k	1.2k	Tools
KickMaterial	1.0	79.1k	1.6k	Crowdfunding
KISS Launcher	3.11.9	27.2k	1.4k	Personalisation
MedLog	1.0	65k	0	Medical
Minimal To Do	1.2	27.5k	1.8k	Productivity
MoneyManagerEx	02.14.994	170k	265	Finance
My Expenses	3.0.7.1	1835.6k	248	Finance
NYBus	1.0	6.9k	272	Transport
Omni Notes	6.0.5	105.9k	2k	Productivity
OpenTasks	1.2.4	448k	724	Productivity
ownCloud	2.14.2	333.7k	2.9k	Productivity
Sunflower	0.1.6	5.3k	10.1k	Gardening
Surveyor	13.3.0	290.4k	13	Communication
WordPress	14.2-rc-2	461.7k	2.3k	Productivity

7.2 Experiment setup

We conduct two studies to answer above research questions. Study 1 addresses RQ1 and RQ2 and study 2 addresses RQ3. For study 1, we evaluate FlakeScanner on GUI tests in FlakyAppRepo (that have been annotated as flaky tests by developers) to check whether FlakeScanner can detect such flaky tests. First, we select all tests in the benchmark that are marked as flaky and exclude tests that are not GUI tests (*e.g.*, tests for database operations), then execute them on Android emulators. The passing tests are used in study 1. In the end, 24 GUI tests are collected from 6 apps, which is shown in Table 2. FlakeScanner and RERUN execute each of them. If a test failure occurs during execution, the flaky test is considered to be successfully detected. The results of each test is recorded for analysis. For study 2, we exclude tests that are used in study 1 and execute the remaining Android instrumentation tests on emulators and the passing tests are selected for evaluation. Eventually, 1444 tests are obtained from the 28 apps and are executed by FlakeScanner. If a test failure is detected, FlakeScanner reports the test as a flaky test.

We conducted experiments on a physical machine with 64 GB RAM and a 56 cores Intel(R) Xeon(R) E5-2660 v4 CPU, running a 64-bit Ubuntu 16.04 operating system. Each execution instance runs in a Docker container to minimize the potential inference between running instances. App under test runs on an Android 9 (x86) emulator. One execution instance is for one test case for which the Android emulator is initialized to a fresh state at the begining to provide a clean testing environment.

7.3 RQ1: Efficacy

Table 2 shows results of FlakeScanner on the data set of known flaky tests. The first column indicates test Ids, the second column shows app names and testing frameworks used in apps, and the third column indicates test method names. Column "#Op" represents how many statements in a test which perform thread synchronization during testing. This is computed by manually counting synchronization operations such as `waitFor()`, `await()` as well as `onView()`, `onData()` in the Espresso framework. Column "#Events" indicates the number of events observed by FlakeScanner during detection. Column "#Run" shows times the number of times a test is executed for flakiness detection. Column "Time" reports the time that is used to detect a flaky test. Column "Ctg" indicates which category the flaky test belongs to in root cause analysis (we identify four categories C1-C3, as mentioned in the following). Column "Succ" indicates whether the test is identified as a flaky test by FlakeScanner.

As we see in Table 2, FlakeScanner successfully detected 19 flaky tests. For test 6, 7, and 8, FlakeScanner could execute them but failed to identify them as flaky tests. Code inspection shows these tests are flaky due to using a unsophisticated synchronization mechanism, i.e., waiting for a fixed amount of time for asynchronization operations. These 3 tests extract meta data for given videos on the internet and specify 10 seconds waiting for accessing the internet. If it takes more than 10 seconds to connect the internet, they will fail. FlakeScanner can monitor thread synchronization between testing frameworks and apps and stops delaying async events once this synchronization occurs. Thus, these tests passed without being identified as flaky tests. For test 11 and 24, FlakeScanner failed to execute them due to configuration issues, e.g., *preview_isShowing* from `CameraView` app passes in API 21 and fails for any of the later APIs: This test usage a view called *TextureView*, which was updated in later Android APIs.

Testing frameworks often provide various mechanisms to avoid test flakiness, e.g., Espresso uses method `onView()` to synchronizes view operations with the UIThread. Tests in Table 2 are developed with such test frameworks. Why are they still flaky? To answer this question, we perform an empirical study on root causes of these flaky tests. The root causes are classified into 3 categories:

Category 1 (C1): Tests are flaky due to non-deterministic execution environments. Apps often interact with background services or resources and exchanges data. For some reason (e.g., being used in other computation), these services or resources may be unable to respond in time, which leads to a `Timeout` exception in the UI thread or testing frameworks and causes a test failure eventually. As shown in Table 2, it is the most common root causes and 12 tests belong to this category. Unfortunately, testing frameworks cannot handle such asynchronism that occurs in the execution environment though they provide mechanisms to synchronize GUI operations or pre-defined resources.

Category 2 (C2): A test expects an implicit event execution order which may not always occur in the execution. Events are not only used for data exchange between threads but also used to perform operations, e.g., an intent is often used to launch an activity. An event execution order change resulted from async threads can lead to a different app behavior such as the soft keyboard disappearing

Table 2: Results on known flaky tests by FlakeScanner and RERUN.

Test Id	App:Framework	Method name	FlakeScanner						RERUN		
			#Op	#Events	#Run	Time(s)	Ctg	Succ	#Run	Time(s)	Succ
1	Surveyor: Espresso	capture	1	58	2	48	C3	✓	2	5	✓
2		twoQuestions	15	104	2	48	C1	✓	20	264	
3		multimedia	9	233	2	158	C2	✓	20	113	
4		contactDetails	4	104	2	48	C1	✓	20	276	
5	Android Youtube Extractor: Junit	testEncipheredVideo	3	4	2	24	C3	✓	2	9	✓
6		testUnembeddable	5	7	3	22	C1		20	23	
7		testAgeRestrictVideo	5	5	3	12	C1		20	21	
8		testUsualVideo	3	5	3	14	C1		20	19	
9	MyExpenses: Espresso	testScenarioForBug5b11072e6007d59fcd92c40b	4	861	2	100	C1	✓	20	153	
10		editCommandKeepsListSize	2	101	2	54	C1	✓	20	168	
11		cloneCommandIncreasesListSize	2	-	-	-	-		20	156	
12		changeOfFractionDigitsWithUpdateShouldKeepTransactionSum	5	991	2	170	C2	✓	20	142	
13		changeOfFractionDigitsWithoutUpdateShouldChangeTransactionSum	5	991	2	174	C2	✓	20	135	
14	Firefox Lite: Espresso	saveImageThenDelete_imageSaveAndDeleteSuccessfully	3	627	2	164	C2	✓	20	230	
15		dismissMenu	3	165	2	76	C1	✓	20	171	
16		turnOnTurboModeDuringOnBoarding_turboModeIsOnInMenu	8	89	2	40	C1	✓	1	7	✓
17		changeDisplayLang	5	189	2	72	C1	✓	3	33	✓
18	AntennaPod: Robotium	testGoToPreferences	1	107	2	56	C1	✓	4	22	✓
19		testClickNavDrawer	7	132	2	60	C1	✓	1	5	✓
20		PlaybackSonicTest#testContinuousPlaybackOnMultipleEpisodes	4	177	2	142	C2	✓	20	254	
21		PlaybackSonicTest#testContinousPlaybackOffMultipleEpisodes	3	176	2	140	C2	✓	20	279	
22		PlaybackTest#testContinousPlaybackOffMultipleEpisodes	3	175	2	142	C2	✓	20	276	
23		PlaybackTest#testContinuousPlaybackOnMultipleEpisodes	4	162	2	140	C2	✓	20	241	
24	CameraView: Espresso	preview_isShowing	1	-	-	-	-	-	20	86	
sum/ave			4.4	248	2	83		19	15	127	6

late, which leads to a test failure. In our study, 8 cases belong to this category.

Category 3 (C3): Flaky tests are caused by data race between the testing thread and threads in apps. In many cases, data that is used to check app behavior by a test is produced asynchronously, i.e., by a background thread. The data can be updated late for sometimes and the test checks "old" data, which lead to a test failure. We have 4 such as cases in our study.

In summary, despite testing frameworks' support to eliminate flakiness, flaky tests still occur due to non-determinism from execution environments (C1 and C2) and developers omitting certain cases resulted from thread concurrency (C3).

7.4 RQ2: Comparison with existing techniques

Comparison with RERUN. RERUN is widely used to examine whether a test is flaky. A failed test is deemed flaky if the test passes in multiple reruns. The approach works in our setting as well. A passing test is deemed flaky if the test fails in multiple reruns. We take RERUN as the base line tool and compare it with FlakeScanner. In our experiment, we rerun a test for 20 times. If a failure is detected, the test is identified as a flaky test and the execution is terminated.

As shown in Table 2, for most cases, FlakeScanner successfully detects a flaky test during the second run. Totally 19 flaky tests are detected and all of them are detected less than 3 minutes. RERUN successfully detects 6 flaky tests. Two of them are detected in the first run. The others are detected in 4 runs. In terms of execution time of a single test, RERUN runs faster than FlakeScanner since FlakeScanner takes time for dynamic analysis. However, FlakeScanner detects much more flaky tests than RERUN.

With regards to the timing comparison with RERUN, note that RERUN was run only 20 times per test. As a result, when we report that RERUN took 127 seconds on average, it is a gross underestimation of the actual time taken by RERUN.

Overall FlakeScanner detects most flaky tests in the second run i.e., identifying event schedule space phase. The experiments also show that our optimization on schedule space identification is effective. Schedule space identification involves maximally delaying an async event, which most likely triggers a test-flaky failure. We compute schedule spaces of multiple async events at the same time. A flaky test most likely fails in this phase. Therefore, this strategy can significantly reduce the number of runs to detect a flaky test.

Comparison with race-detection techniques. Event race detection techniques DROIDRACER [25], ERVA [18], EVENTRACER [7], and CAFA [17] run on a modified Android framework 4.3 or 4.4. We face incompatibility issues to evaluate those techniques on the collected apps because many apps target Android frameworks with a higher version (we use Android framework 9.0 in the evaluation). Therefore, we perform a qualitative comparison analysis between FlakeScanner and race-detection techniques.

False positives. Flaky test detection leveraging data race detection techniques will depend on an accurate computation of the happens-before relation. However, data race detection techniques capture happen-before relations by monitoring event operations in UIThread. Event dependencies due to synchronization between testing framework (like Espresso) and UIthread (app under test) will not be captured. This leads to an underestimation of the happens-before relation. As an example event e1 can denote a button appearing in the screen due to an async thread completing computation, event e2 can be a button click in testing framework, and e1 happens-before e2. Such happens-before edges are dropped by race detectors. Flaky test detection leveraging data race detectors may lead to many false positives among the flaky tests reported. In contrast, FlakeScanner

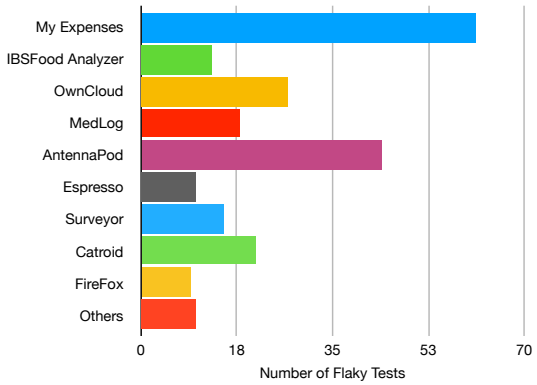


Figure 4: Flaky test distribution.

can precisely detect synchronization between testing framework and UIthread to avoid such false positives.

Maintainability. Race-detection techniques often require a modified Android framework to capture happen-before relations and can struggle from fast evolution of Android frameworks. By contrast, FlakeScanner can be used in different Android framework versions since FlakeScanner requires no modification to Android framworks and the debug mode that FlakeScanner relies on is supported by most Android frameworks.

7.5 RQ3: Real-world flaky detection

To validate the effectiveness of FlakeScanner on discovering new flaky tests, we ran FlakeScanner on 1444 tests in FlakyAppRepo which are not marked as flaky. The results show FlakeScanner is effective in discovering new flaky tests. Out of 1444 tests, FlakeScanner successfully detected 245 flaky tests. Figure 4 shows distribution of the detected flaky tests among apps. FlakeScanner discovered the most flaky tests in app My Expenses (61 flaky tests) and less flaky tests in FireFox. We also collected statistics on error messages of the failures of these flaky tests, which is shown in Table 3. The most common error message is "Waited for the root of the view hierarchy to have window focus" and the next is "No views in hierarchy found matching". In other words, most failures are related to mismatch between GUI operations and app state.

7.5.1 *Manually Checking Ground Truth*

To further validate the detected results, we manually investigated 20 randomly selected cases among 245 reported tests. This is to manually check whether the tests are actually flaky.

The investigation shows these tests usually pass, however, they fail only for one or some corner cases. We leveraged the event orderings discovered by FlakeScanner to identify these corner cases. Two of the authors then studied the effect of those event orders and verified that these tests were indeed flaky. As it turns out during this manual process the two authors were in agreemnt and there was no disagreement which needed to be resolved. We then reported these cases to the corresponding developers with the detailed reports on how to reproduce them.

At the time of writing of the paper, we got 11 out of the 20 test cases confirmed as flaky tests. Five tests are still under investigation by developers, and we did not hear back from developers for four tests.

Table 3: Common Exceptions and their frequencies

Reason	#
Waited for the root of the view hierarchy to have window focus	97
No views in hierarchy found matching	36
Could not inject Intent	25
Assertion Failed Error	19
Error performing single click: Animations are enabled on the target device	15
Unexpected state change	12
No such file or directory	8
java.lang.NoClassDefFoundError: android.support.test.espresso.intent.Intents	6
TembaException: Unable to fetch **	6
Attemp to invoke virtual method on a null reference object	4
MediaPlayer error	4
Wait for [**] to become idle timed out	4
others	9

7.5.2 *Case Study: FirefoxLite– SwitchSearchEngineTest*

Figure 5 shows parts of code snippet from the *SwitchSearchEngineTest* test for the *FirefoxLite* app. It tests the functionality provided by a broadcast receiver *SearchEngineManager*, which initializes and loads different search engines. During the startup process, search engines are loaded by *loadSearchEngines* method. This method creates a worker thread (*SearchEngines-Load*) to initiate the loading process in the background. Loading of search engines is verified (in main thread) via *awaitLoadingSearchEnginesLocked*.

Evidently, we have three asynchronous threads: the espresso thread, the UIthread, and the worker thread. Worker threads in Android are implicitly moved into a background control group (*cgroup*), where they only get a small percentage of the available CPU [16]. In the scenario, where the worker thread has not started (waiting to get scheduled) and the espresso thread tries to access the default search engine, the above test fails. This depends on various factors such as the percentage of the CPU available and the number of background threads running. FlakeScanner detected the test as a flaky test by exploring different event execution orders. Since there is no synchronization between the testing thread and the worker thread, FlakeScanner delayed the worker thread at event schedule space identification phase such that the test failed and was reported the test as a flaky test.

8 LIMITATIONS

We identify the following potential limitations to our evaluation.

- *Event identification.* In our system, an event identifier is generated with data from two different threads. We may not successfully hook the event with its identifier for some rare cases, e.g., when our system runs extremely slowly, two pieces of data may not match the event identifier at the same time. Addressing this, we run our experiments on a system with a light workload and configure an emulator with a larger amount of memory (i.e., 8G).
- *Thread dependency resolution.* In event schedule space identification, when too many threads are suspended at the same time, we terminate the execution and resolve thread dependencies to avoid imprecise schedule space identification.

```
public class SwitchSearchEngineTest {
//…
private final List<SearchEngine> searchEngines =
SearchEngineManager.getInstance().getSearchEngines();
//…
@Test
public void switchSearchEngine_searchViaSearchEngineAccordingly() {
activityTestRule.launchActivity(new Intent());
final SearchEngine defaultSearchEngine =
SearchEngineManager.getInstance().getDefaultSearchEngine(…)
//…
}


public class SearchEngineManager extends BroadcastReceiver {
private boolean loadHasBeenTriggered = false;

public void loadSearchEngines(final Context context) {
    new Thread("SearchEngines-Load") {
        @Override
         public void run() {
             loadFromDisk(context);  ➡  loadHasBeenTriggered = true
         }
     }.start();
}


public void awaitLoadingSearchEnginesLocked() {

if (!loadHasBeenTriggered) {
    throw new IllegalStateException(…);
}
```

Figure 5: SwitchSearchEngineTest

- *Empirical study.* During our manual analysis on flaky tests, at least two of the authors analyze the log of a test failure for each flaky test to ensure the root cause of the test is correctly understood.

9 RELATED WORK

Flaky test detection and fixing. A few earlier researchers have started to work on flaky test issues. Bell et al. [5] use code coverage differential analysis to identify flaky tests. A test is deemed flaky if it fails in the regression testing and its execution does not reach any code that was recently changed by developers. Shi et al. [27] propose an approach to fix order-dependent flaky tests by leveraging passing tests. Shi et al. [26] propose to rerun a test multiple times on each mutant and obtain reliable coverage results such that the effects of flaky tests on mutation testing can be mitigated. Different from them, FlakeScanner detects concurrency-related flaky tests in Android apps by exploring feasible event execution orders.

Event race detection. Another branch of works that are close to ours is event race detection. Instead of detecting flaky tests, they leverage dynamic and static analysis to detect harmful event race. For instance, DROIDRACER [25], ERVA [18], EVENTRACER [7], CAFA [17], and nAdroid [15] capture *happens-before-relation* among events and inference possible event race errors. In addition, Ozkan et al. [21] propose to detect asynchronous bugs by exploring different execution orders of event handlers in Android apps. These techniques have potential to apply to flaky test detetion, but face challenges to capture complete and precise happpen-before relations when a test is executed by a testing framework like Espresso. Many false positives can be reported due to incomplete happen-before relations as explained in Section 7.4. By contrast, FlakeScanner performs a system-level dynamic analysis to capture precise event dependencies to avoid such false positives.

Empirical studies on flaky tests. Multiple studies [12, 24, 28] confirm concurrency as the major cause of flaky tests. Luo et al. [24] performed an empirical analysis of flaky tests in 51 open-source projects. They identified *Concurrency* and *Async wait* as the most common cause of flaky tests. They pointed out that the majority of these cases arose because they do not wait for external resources. Finally, they described the common fixing strategies the developers use to fix flaky tests. In a separate study, Eck et al.[12] surveyed 21 professional developers to classify 200 flaky tests they fixed. They identified four unreported causes of flaky tests, which are also considered difficult to fix. Thorve et al. [28] conducted an empirical study of flaky tests in Android apps. They searched 1000 projects for the commits related to flakiness and found only 77 relevant commits from 29 projects. They found 36% of commits occurred due to concurrency related issues. Fan et al. [14] proposed a hybrid approach towards manifesting asynchronous bugs in Android apps. They studied 2097 apps and identified three async programming rules implied by the single-GUI-thread model. Based on these rules, they categorized three fault pattern and used static analysis to locate them in the app. Subsequently, they map these program traces to real event sequences to verify these errors.

Concurrency bugs detection. There have been several testing based approaches [9, 11, 19, 20, 22, 31] to identify concurrency related bugs. Maple [31] proposed a coverage-driven approach to expose untested thread interleavings. Letko [22] proposed a combination of testing and dynamic analysis with *metaheuuristic* techniques. Choudhary et al. [9] presented a coverage-guided approach for generating concurrency tests to detect bugs in thread-safe classes. Multiple related works [3, 6, 8, 13, 23, 29] manipulated event orders to control non-determinism in multi-threaded programs. Liu et al. [23] proposed a deterministic multithreading system that replaces *pthreads* library in C/C++ apps. Emmi et al. [13] proposed a search prioritization strategy to discover concurrency bugs. They add non-determinism to deterministic schedulers by delaying their next-scheduled task. Adamsen et al. [3] presented an automated program repair technique for event race errors in JavaScript. Given a repair policy, they controlled the event handler scheduling in the browser to avoid bad orderings.

10 CONCLUSION

Flaky tests pose a significant problem in validating mobile apps. Recent studies [12, 24, 28] have shown concurrency as the most common cause of flaky tests. The uncertainty in a test outcome may arise due to synchronization issues originating from multiple threads interacting in a undesirable manner. In this paper, we presented an approach for detecting flaky tests through a systematic exploration of event orders. We introduced FlakeScanner, a tool to detect flaky tests for Android apps. FlakeScanner explores the space of all realizable execution environments where relevant threads interleave differently.

Due to the lack of a testing benchmark for flaky tests, we created the first subject-suite FlakyAppRepo that is used to study GUI test flakiness. FlakyAppRepo contains 28 widely-used Android apps with 2.5k stars on average in GitHub. We applied FlakeScanner to tests from FlakyAppRepo. Results show that FlakeScanner not only detected known flaky tests but also reported 245 new flaky tests. We believe that our tool and results hold out promise for the problem of tackling flaky tests, which is a significant pain point in industrial practice.

REFERENCES

[1] 2020. Espresso. https://developer.android.com/training/testing/espresso

[2] 2021. Flaky Tests at Google and How We Mitigate Them. https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html

[3] C. Q. Adamsen, A. Møller, R. Karim, M. Sridharan, F. Tip, and K. Sen. 2017. Repairing Event Race Errors by Controlling Nondeterminism. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. 289–299. https://doi.org/10.1109/ICSE.2017.34

[4] Nadia Alshahwan, Xinbo Gao, Mark Harman, Yue Jia, Ke Mao, Alexander Mols, Taijin Tei, and Ilya Zorin. 2018. Deploying Search Based Software Engineering with Sapienz at Facebook. In *Search-Based Software Engineering - 10th International Symposium, SSBSE 2018, Montpellier, France, September 8-9, 2018, Proceedings*. 3–45. https://doi.org/10.1007/978-3-319-99241-9_1

[5] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov. 2018. DeFlaker: Automatically Detecting Flaky Tests. In *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*. 433–444.

[6] Tom Bergan, Luis Ceze, and Dan Grossman. 2013. Input-covering schedules for multithreaded programs. *ACM SIGPLAN Notices* 48 (11 2013), 677–692. https://doi.org/10.1145/2544173.2509508

[7] Pavol Bielik, Veselin Raychev, and Martin Vechev. 2015. Scalable Race Detection for Android Applications. *ACM SIGPLAN Notices* 50 (10 2015), 332–348. https://doi.org/10.1145/2858965.2814303

[8] Ahmed Bouajjani, Michael Emmi, Constantin Enea, Burcu Kulahcioglu Ozkan, and Serdar Tasiran. 2017. *Verifying Robustness of Event-Driven Asynchronous Programs Against Concurrency*.

[9] Ankit Choudhary, Shan Lu, and Michael Pradel. 2017. Efficient Detection of Thread Safety Violations via Coverage-Guided Generation of Concurrent Tests. In *IEEE/ACM International Conference on Software Engineering (ICSE)*. 266–277.

[10] Shauvik Roy Choudhary, Alessandra Gorla, and Alessandro Orso. 2015. Automated Test Input Generation for Android: Are We There Yet? (E). In *Proceedings of the 2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE) (ASE '15)*. 429–440. https://doi.org/10.1109/ASE.2015.89

[11] James Davis, Arun Thekumparampil, and Dongyoon Lee. 2017. Node.fz: Fuzzing the Server-Side Event-Driven Architecture. In *European Conference on Computer Systems (Eurosys)*. 145–160.

[12] Moritz Eck, Fabio Palomba, Marco Castelluccio, and Alberto Bacchelli. 2019. Understanding flaky tests: the developer's perspective. In *27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 830–840.

[13] Michael Emmi, Shaz Qadeer, and Zvonimir Rakamaric. 2011. Delay-Bounded Scheduling. In *Proceedings of Symposium on Principles of Programming Languages (POPL)*. 411–422.

[14] Lingling Fan, Ting Su, Sen Chen, Guozhu Meng, Yang Liu, Lihua Xu, and Geguang Pu. 2018. Efficiently Manifesting Asynchronous Programming Errors in Android Apps. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE)*.

[15] Xinwei Fu, Dongyoon Lee, and Changhee Jung. 2018. nAdroid: statically detecting ordering violations in Android applications. In *Proceedings of the 2018 International Symposium on Code Generation and Optimization (CGO)*. 62–74.

[16] Anubhav Gupta. 2018. Exploring Android Thread Priority. https://medium.com/mindorks/exploring-android-thread-priority-5d0542eebbd1

[17] Chun-Hung Hsiao, Jie Yu, Satish Narayanasamy, Ziyun Kong, Cristiano Pereira, Gilles Pokam, Peter Chen, and Jason Flinn. 2014. Race Detection for Event-Driven Mobile Applications. *ACM SIGPLAN Notices* 49 (06 2014). https://doi.org/10.1145/2666356.2594330

[18] Yongjian Hu, Iulian Neamtiu, and Arash Alavi. 2016. Automatically verifying and reproducing event-based races in Android apps. In *International Symposium on Software Testing and Analysis (ISSTA)*. 377–388.

[19] Casey Klein, Matthew Flatt, and Robert Findler. 2010. Random Testing for Higher-Order, Stateful Programs. In *Proceedings of the Conference on Object-Oriented Programming Systems, Languages, and Applications, OOPSLA*. 555–566.

[20] Bohuslav Krena, Zdenek Letko, Tomas Vojnar, and Shmuel Ur. 2010. A platform for search-based testing of concurrent software. In *PADTAD 2010 - International Workshop on Parallel and Distributed Systems: Testing, Analysis, and Debugging*. 48–58.

[21] Burcu Kulahcioglu Ozkan, Michael Emmi, and Serdar Tasiran. 2015. Systematic Asynchrony Bug Exploration for Android Apps. In *International Conference on Computer Aided Verification (CAV)*. 455–461.

[22] Zdeněk Letko. 2013. Analysis and Testing of Concurrent Programs. *Information Sciences and Technologies Bulletin of the ACM Slovakia* 5, 3 (2013), 1–8. https://www.fit.vut.cz/research/publication/10494

[23] Tongping Liu, Charlie Curtsinger, and Emery Berger. 2011. Dthreads: Efficient deterministic multithreading. In *SOSP'11 - Proceedings of the 23rd ACM Symposium on Operating Systems Principles*. 327–336.

[24] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An empirical analysis of flaky tests. In *International Symposium on Foundations of Software Engineering (FSE)*. 643–653.

[25] Pallavi Maiya, Aditya Kanade, and Rupak Majumdar. 2014. Race Detection for Android Applications. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Edinburgh, United Kingdom) *(PLDI '14)*. Association for Computing Machinery, New York, NY, USA, 316–325. https://doi.org/10.1145/2594291.2594311

[26] August Shi, Jonathan Bell, and Darko Marinov. 2019. Mitigating the effects of flaky tests on mutation testing. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 112–122.

[27] August Shi, Wing Lam, Reed Oei, Tao Xie, and Darko Marinov. 2019. iFixFlakies: a framework for automatically fixing order-dependent flaky tests. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC-FSE)*. 545–555.

[28] Swapna Thorve, Chandani Sreshtha, and Na Meng. 2018. An Empirical Study of Flaky Tests in Android Apps. In *International Conference on Software Maintenance and Evolution (ICSME)*. 534–538.

[29] Ermenegildo Tomasco, Omar Inverso, Bernd Fischer, Salvatore La Torre, and Gennaro Parlato. 2015. Verifying Concurrent Programs by Memory Unwinding. In *21st International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*.

[30] Wenyu Wang, Dengfeng Li, Wei Yang, Yurui Cao, Zhenwen Zhang, Yuetang Deng, and Tao Xie. 2018. An Empirical Study of Android Test Generation Tools in Industrial Cases. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering* (Montpellier, France) *(ASE 2018)*. Association for Computing Machinery, New York, NY, USA, 738–748. https://doi.org/10.1145/3238147.3240465

[31] Jie Yu, Satish Narayanasamy, Cristiano Pereira, and Gilles Pokam. 2012. Maple: A Coverage-Driven Testing Tool for Multithreaded Programs. In *Proceedings of the Conference on Object-Oriented Programming Systems, Languages, and Applications, OOPSLA*. 485–502.